



Venom: A PyTorch Generative Modeling Toolkit

Liang Yan

Paul G. Allen School of Computer Science & Engineering

University of Washington

<https://divinyan.com/Venom/>

<https://github.com/yanliang3612/Venom>

Abstract

Modern generative modeling has grown into a broad collection of related but often separately implemented paradigms, including denoising diffusion models, score-based stochastic differential equations, flow matching, variational autoencoders, normalizing flows, adversarial models, and energy-based models. For newcomers, this fragmentation makes it difficult to compare training objectives, inference procedures, sampling algorithms, and conditioning mechanisms within a single coherent codebase. We introduce VENOM, an educational PyTorch toolkit that implements representative generative modeling families under a unified, MNIST-first interface. VENOM emphasizes breadth, readability, reproducible entry points, and consistent training and sampling APIs rather than large-scale performance engineering. The package currently includes diffusion and score-based models, flow matching and one-step generators, variational autoencoders, normalizing flows, generative adversarial networks, and energy-based models. It provides separate training and sampling scripts, classifier and classifier-free guidance examples, bilingual tutorial notebooks, and a model-family organization that supports teaching, prototyping, and lightweight benchmarking.

1 Introduction

Generative modeling has become a central area of modern machine learning. Diffusion models [4, 19, 33], score-based models [40, 41], variational autoencoders [23], normalizing flows [6, 37], generative adversarial networks [10], and energy-based models [7, 25] provide different ways to learn, sample from, and condition on complex data distributions. Although these families are often studied separately, they address closely related questions: how should probability distributions be represented, how should samples be generated, and how should conditioning or guidance be incorporated?

This diversity has greatly advanced the field, but it also creates a practical barrier for learning, comparison, and prototyping. Research code is often designed around a specific paper or model family, while tutorials typically focus on one paradigm at a time. As a result, newcomers may learn the mechanics of diffusion training without seeing how it relates to adversarial learning, exact-likelihood normalizing flows, variational inference, or energy-based sampling. Similarly, researchers and instructors who want a compact benchmark or teaching scaffold often need to combine multiple repositories with different APIs, dataset conventions, training scripts, and checkpoint formats.

VENOM is designed to address this gap. It collects representative algorithms from major generative modeling families into a single lightweight PyTorch codebase with a unified, MNIST-first interface. Rather than aiming to reproduce every engineering detail of large-scale image generation systems, VENOM emphasizes breadth, readability, and consistency. Its goal is to provide a compact and inspectable map of common training objectives, inference procedures, sampling algorithms, and guidance mechanisms. By using MNIST

as a shared starting point, each model family can be trained quickly, inspected easily, and extended to more complex datasets or research settings.

2 Design Goals

VENOM is designed around four principles: clear organization, consistent entry points, educational coverage, and lightweight benchmark readiness.

Family-level organization. Each major generative modeling paradigm is implemented in a dedicated sub-package, including `venom.diffusion`, `venom.vae`, `venom.flows`, `venom.gan`, and `venom.ebm`. This structure makes the connection between the codebase and the underlying modeling families explicit, allowing users to navigate the repository by conceptual paradigm rather than by implementation details.

Consistent training and sampling entry points. We standardize the experimental interface across model families by providing paired training and sampling scripts for each method. For example, diffusion models expose `train_diffusion.py` and `sample_diffusion.py`, while normalizing flows expose `train_flow.py` and `sample_flow.py`. This consistent structure reduces the overhead of reproducing experiments and makes it easier to compare different model families under the same workflow. Users can therefore run meaningful experiments through simple command-line entry points without first learning a large or highly specialized framework.

Educational completeness. The package intentionally covers a broad range of training and inference paradigms, including variational lower bounds, adversarial objectives, exact likelihood maximization, score matching, contrastive divergence, Langevin sampling, ODE/SDE-based sampling, and classifier guidance. The goal is not only to provide working implementations, but also to help users understand how these paradigms relate to one another within a shared experimental setting.

Benchmark readiness. Because the implemented families share a default dataset, checkpoint convention, and sampling interface, VENOM can also serve as a lightweight benchmark for comparing objectives, samplers, and conditioning mechanisms. This makes the package useful not only for learning, but also for controlled prototyping and small-scale empirical comparison.

3 Implemented Model Families

Table 1 summarizes the current model coverage in VENOM. The package does not aim to provide state-of-the-art reproductions of every original method. Instead, it offers compact and inspectable implementations of the core ideas behind each model family, making them suitable for learning, prototyping, and controlled benchmarking.

4 Package Structure and API

VENOM provides both command-line entry points and Python APIs. The command-line interface is designed for quick experiments and reproducible examples, while the Python API supports extension, modification, and research reuse.

Table 1: Model families covered by VENOM.

Family	Implemented variants	Core idea represented
Diffusion and score-based models	DDPM, Improved DDPM, ADM, CFG, DDIM, DPM-Solver, Score SDE, NCSN, EDM, PFGM	Denoising objectives, score estimation, SDE/ODE sampling, and classifier or classifier-free guidance [4, 19, 21, 29, 33, 39–41, 45].
Flow matching and one-step models	Rectified Flow, Flow Matching, Conditional Flow Matching, stochastic interpolants, consistency models, shortcut models, MeanFlow	Velocity-field learning, trajectory-based generation, and fast few-step or one-step generation, including consistency-style distillation [9, 27, 28, 38, 42, 43].
Variational autoencoders	VAE, ConvVAE, β -VAE, CVAE, IWAE, VQ-VAE, Ladder VAE, Flow-VAE	Latent-variable modeling, amortized inference, importance weighting, discrete latent representations, and flow-based posterior corrections [2, 15, 23, 37, 44].
Normalizing flows	Planar, radial, NICE, RealNVP, Glow, MAF, IAF, Neural Spline Flow, FFJORD, Flow++	Invertible transformations, change-of-variables likelihoods, and exact density estimation [5, 6, 8, 11, 18, 22, 24, 35, 37].
Generative adversarial networks	GAN, DCGAN, CGAN, AC-GAN, InfoGAN, LSGAN, WGAN, WGAN-GP, HingeGAN, SNGAN	Adversarial learning, conditional generation, Wasserstein objectives, gradient penalties, hinge-loss training, and spectral normalization [1, 3, 10, 13, 26, 30–32, 34, 36].
Energy-based models	RBM, Gaussian RBM, Conditional RBM, ConvRBM, Deep EBM, Conditional EBM, JEM, score matching EBM, sliced score matching EBM, NCE-EBM	Explicit energy functions, contrastive divergence, Langevin/SGLD sampling, score matching, and noise-contrastive estimation [7, 12, 14, 16, 20].

Listing 1: Example command-line workflow.

```

pip install -e .
python train_diffusion.py --variant ddpm --epochs 5
python sample_diffusion.py --checkpoint runs/mnist_diffusion/ddpm/model_005.pt

python train_vae.py --variant conv-vae --epochs 5
python train_flow.py --variant realnvp --epochs 5
python train_gan.py --variant dcgan --epochs 5
python train_ebm.py --variant rbm --epochs 5

```

The Python API follows a family-specific factory pattern:

Listing 2: Example Python API usage.

```

from venom.diffusion.factory import build_mnist_diffusion
from venom.flows import build_mnist_flow
from venom.gan import build_mnist_gan

model, diffusion = build_mnist_diffusion("ddpm")
flow, flow_config = build_mnist_flow("realnvp")
generator, discriminator, gan_config = build_mnist_gan("dcgan")

```

This design keeps the modeling assumptions visible rather than hiding them behind a single generic

interface. For example, normalizing flows expose likelihood-oriented methods such as `log_prob`; diffusion models expose different sampler choices such as DDPM, DDIM, and DPM-Solver; and EBMs expose Gibbs, Langevin, or SGLD-style sampling procedures. As a result, users can interact with each family through an interface that reflects its underlying mathematical structure while still benefiting from a consistent repository-level workflow.

5 Training, Inference, and Guidance Paradigms

A central goal of VENOM is to place different generative modeling paradigms within the same codebase and make their operational differences explicit.

Training. Different model families rely on different training principles. Diffusion models train denoisers or score networks; VAEs optimize variational bounds; normalizing flows maximize exact likelihoods under invertible transformations; GANs optimize adversarial objectives; and EBMs learn scalar energy functions using contrastive, score-based, or noise-contrastive estimators. These objectives are often introduced in separate literatures, but in VENOM they can be studied with comparable data loaders, training loops, checkpoint structures, and sampling outputs.

Inference and sampling. Sampling also differs substantially across model families. Diffusion models reverse a stochastic or deterministic denoising process; score SDEs use predictor-corrector or ODE-based samplers; normalizing flows transform Gaussian latent variables through an invertible map; GANs generate samples through a single forward pass; VAEs decode latent variables; and EBMs use MCMC-style Gibbs, Langevin, or SGLD transitions. VENOM exposes these differences through separate sampling scripts and family-specific APIs, making the computational behavior of each paradigm easier to inspect and compare.

Guidance and conditioning. The package includes class conditioning, classifier guidance, and classifier-free guidance for diffusion-style models [4, 17]. It also includes label-conditioned variants of VAEs, GANs, and EBMs. This allows users to compare modern guidance mechanisms with simpler conditional generation paradigms in a shared experimental environment.

6 Tutorials and Benchmark Use

The repository includes both Chinese and English Jupyter notebooks that walk through training, sampling, guidance, checkpoint locations, and smoke-test performance indicators. These tutorials are intended to make the package accessible to entry-level users, students, and researchers who want a compact overview of modern generative modeling methods.

The same scripts can also be used as a small benchmark suite. Each model family writes checkpoints and sample grids under a separate `runs/` directory, which makes it straightforward to compare training behavior, sampling behavior, and qualitative outputs across different objectives and samplers. Although VENOM is not designed as a large-scale benchmark, its unified interface makes it useful for lightweight empirical studies and teaching-oriented comparisons.

7 Scope and Limitations

VENOM is not intended to replace specialized high-performance implementations. Large-scale diffusion systems, high-resolution flow models, and industrial GAN training pipelines require additional engineering,

data infrastructure, distributed training, and evaluation tools. Instead, VENOM targets the gap between isolated tutorials and large research repositories. Its emphasis is on coverage of core paradigms, conceptual clarity, readability, and ease of modification.

The current implementation is centered on MNIST to keep experiments fast and accessible. While this choice makes the package suitable for education, debugging, and controlled comparison, it also means that the default setting is not intended to measure large-scale generative modeling performance. Future extensions could include larger datasets, stronger evaluation protocols, and more advanced architectures while preserving the same family-level organization and consistent training interface.

8 Conclusion

We presented VENOM, a PyTorch toolkit that brings major generative modeling paradigms into a single educational and benchmark-friendly codebase. By implementing representative diffusion, score-based, flow matching, VAE, normalizing flow, GAN, and EBM methods with consistent training and sampling interfaces, VENOM helps beginners understand the landscape of modern generative modeling and provides researchers with a compact scaffold for experimentation, extension, and lightweight comparison.

Software Availability

The project page is available at <https://divinyan.com/Venom/>. The source code is available at <https://github.com/yanliang3612/Venom>.

Acknowledgments

This software paper draft describes the open-source VENOM project.

References

- [1] Martin Arjovsky, Soumith Chintala, and Leon Bottou. Wasserstein generative adversarial networks. In *International Conference on Machine Learning*, 2017.
- [2] Yuri Burda, Roger Grosse, and Ruslan Salakhutdinov. Importance weighted autoencoders. In *International Conference on Learning Representations*, 2016.
- [3] Xi Chen, Yan Duan, Rein Houthoofd, John Schulman, Ilya Sutskever, and Pieter Abbeel. InfoGAN: Interpretable representation learning by information maximizing generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2016.
- [4] Prafulla Dhariwal and Alexander Quinn Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems*, 2021.
- [5] Laurent Dinh, David Krueger, and Yoshua Bengio. NICE: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.
- [6] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real NVP. In *International Conference on Learning Representations*, 2017.

- [7] Yilun Du and Igor Mordatch. Implicit generation and modeling with energy based models. In *Advances in Neural Information Processing Systems*, 2019.
- [8] Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural spline flows. In *Advances in Neural Information Processing Systems*, 2019.
- [9] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*, 2025.
- [10] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.
- [11] Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FJORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2019.
- [12] Will Grathwohl, Kuan-Chieh Wang, Jörn-Henrik Jacobsen, David Duvenaud, Mohammad Norouzi, and Kevin Swersky. Your classifier is secretly an energy based model and you should treat it like one. In *International Conference on Learning Representations*, 2020.
- [13] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein GANs. In *Advances in Neural Information Processing Systems*, 2017.
- [14] Michael Gutmann and Aapo Hyvärinen. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *International Conference on Artificial Intelligence and Statistics*, 2010.
- [15] Irina Higgins, Loic Matthey, Arka Pal, Christopher P. Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations*, 2017.
- [16] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.
- [17] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021.
- [18] Jonathan Ho, Xi Chen, Aravind Srinivas, Yan Duan, and Pieter Abbeel. Flow++: Improving flow-based generative models with variational dequantization and architecture design. In *International Conference on Machine Learning*, 2019.
- [19] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.
- [20] Aapo Hyvärinen. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709, 2005.
- [21] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, 2022.
- [22] Diederik P. Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. In *Advances in Neural Information Processing Systems*, 2018.

- [23] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.
- [24] Diederik P. Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. In *Advances in Neural Information Processing Systems*, 2016.
- [25] Yann LeCun, Sumit Chopra, Raia Hadsell, Marc’Aurelio Ranzato, and Fu Jie Huang. A tutorial on energy-based learning. In *Predicting Structured Data*. MIT Press, 2006.
- [26] Jae Hyun Lim and Jong Chul Ye. Geometric GAN. *arXiv preprint arXiv:1705.02894*, 2017.
- [27] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- [28] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.
- [29] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *Advances in Neural Information Processing Systems*, 2022.
- [30] Xudong Mao, Qing Li, Haoran Xie, Raymond Y. K. Lau, Zhen Wang, and Stephen Paul Smolley. Least squares generative adversarial networks. In *International Conference on Computer Vision*, 2017.
- [31] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*, 2014.
- [32] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations*, 2018.
- [33] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, 2021.
- [34] Augustus Odena, Christopher Olah, and Jonathon Shlens. Conditional image synthesis with auxiliary classifier gans. In *International Conference on Machine Learning*, 2017.
- [35] George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation. In *Advances in Neural Information Processing Systems*, 2017.
- [36] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *International Conference on Learning Representations*, 2016.
- [37] Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning*, 2015.
- [38] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations*, 2022.
- [39] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.

- [40] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, 2019.
- [41] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.
- [42] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning*, 2023.
- [43] Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrod Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024.
- [44] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.
- [45] Yilun Xu, Ziming Liu, Max Tegmark, and Tommi Jaakkola. Poisson flow generative models. In *Advances in Neural Information Processing Systems*, 2022.